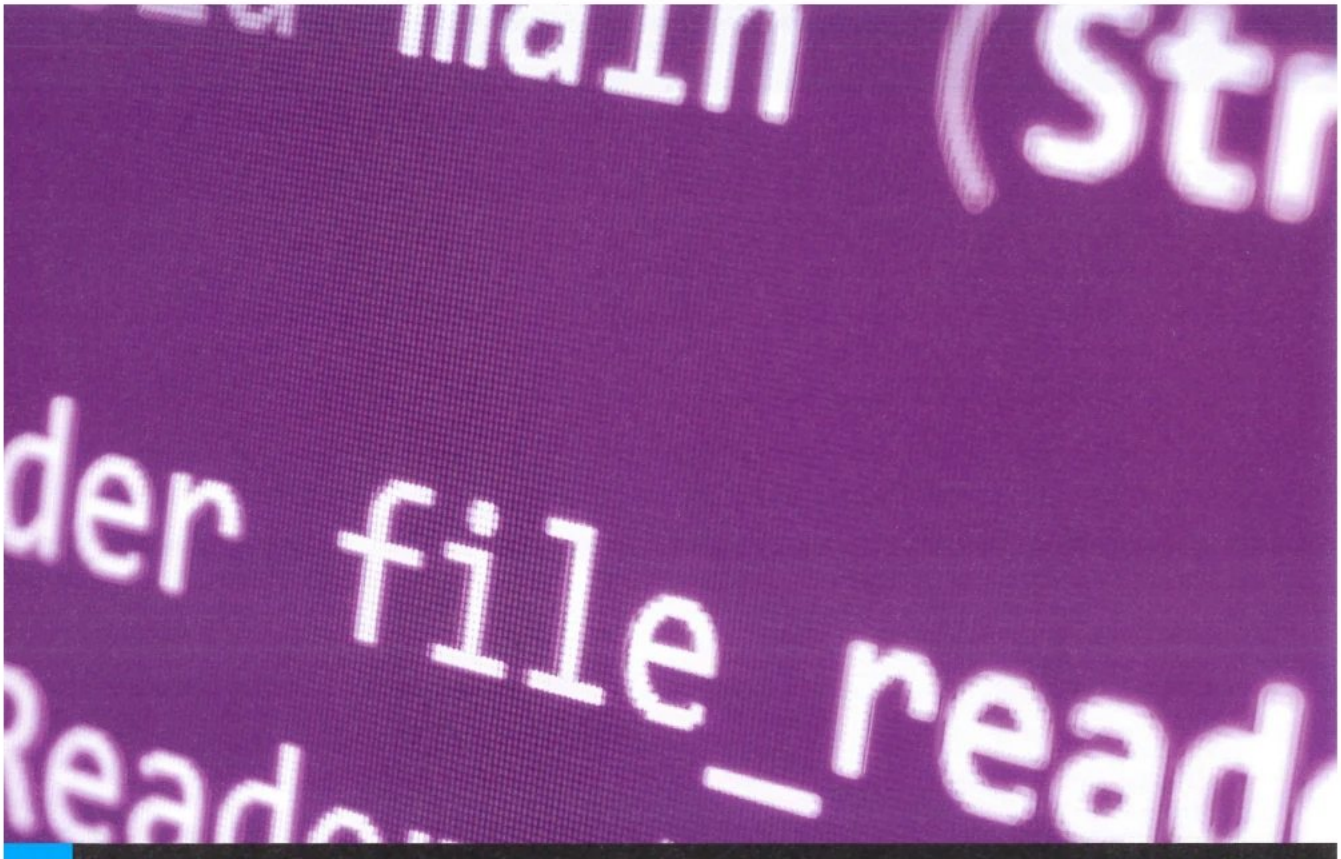


Chapter 26:
File processing and exception handling

Learning objectives

By the end of this chapter you should be able to:

- write code to perform file-processing operations:
  - open a file (in read, write or append mode)
  - read a record from a file
  - write a record to a file
  - on serial, sequential and random files
- show understanding of an exception and the importance of exception handling
- show understanding of when it is appropriate to use exception handling
- write code to use exception handling.



26.01 Records

Records are user-defined types (discussed in Chapter 16, Section 16.01). See also Section 13.02.

WORKED EXAMPLE 26.01

Using records

A car manufacturer and seller wants to store details about cars. These details can be stored in a record structure:

```
TYPE CarRecord
    DECLARE VehicleID           : STRING  // unique identifier and record key
    DECLARE Registration        : STRING
    DECLARE DateOfRegistration  : DATE
    DECLARE EngineSize          : INTEGER
    DECLARE PurchasePrice       : CURRENCY
ENDTYPE
```

To declare a variable of that type we write:

```
DECLARE ThisCar : CarRecord
```

Note that we can declare arrays of records. If we want to store the details of 100 cars, we declare an array of type `CarRecord`

```
DECLARE Car : ARRAY[1:100] OF CarRecord
```


Python

Python does not have a record type. However, we can use a class definition with only a constructor to assign initial values. (See Chapter 27 for more about classes).
The pseudocode example of a car record described in Worked Example 26.01 can be programmed as follows:

```
class CarRecord:          # declaring a class without other methods
   def __init__(self):   # constructor
       self.VehicleID = ""
       self.Registration = ""
       self.DateOfRegistration = None
       self.EngineSize = 0
       self.PurchasePrice = 0.00
ThisCar = CarRecord() # instantiates a car record
ThisCar.EngineSize = 2500 # assign a value to a field
Car = [CarRecord() for i in range(100)] # make a list of 100 car records
Car[1].EngineSize = 2500 # assign value to a field of the 2nd car in list
```

VB.NET

```
Structure CarRecord
   Dim VehicleID As String
   Dim Registration As String
   Dim DateOfRegistration As Date
   Dim EngineSize As Integer
   Dim PurchasePrice As Decimal
End Structure
Dim ThisCar As CarRecord    ' declare a variable of CarRecord type
Dim Car(100) As CarRecord   ' declare an array of CarRecord type
ThisCar.EngineSize = 2500   ' assign value to a field
Car(2).EngineSize = 2500    ' assign value to a field of 2nd car in array
```

(Continued)

Java

Java does not have a record type. However, we can use a data structure. A class definition without methods is a data structure that can be used like a record. (See Chapter 27 for more about classes).
The pseudocode example of a car record described in Worked Example 26.01 can be programmed as follows:

```
class CarRecord
{
   String vehicleID;
   String registration;
   String dateOfRegistration;
   int engineSize;
   double purchasePrice;

   public CarRecord()   // declare a constructor without other methods
   {
      vehicleID = "XX";
      registration = "";
      dateOfRegistration = "01/01/2010";
      engineSize = 0;
      purchasePrice = 0.00;
   }
}

CarRecord thisCar = new CarRecord(); // instantiates a car record
thisCar.engineSize = 2500; // assign a value to a field

CarRecord[] car = new CarRecord[100]; // declare an array of car record type

car[2] = new CarRecord(); // instantiate a car
car[2].engineSize = 2500; // assign a value to a field of 2nd car in array
```


26.02 File processing

In Chapter 13 (Section 13.06) we used text files to store and read lines of text. Text files only allow us to write strings in a serial or sequential manner. We can append strings to the end of the file.

When we want to store records in a file, we create a binary file (see Chapter 16, Section 16.02). We can store records serially or sequentially. We can also store records using direct access to a **random file**. Table 26.01 lists the operations we use for processing files.



KEY TERM

Random file: a file that stores records at specific addresses that can be accessed directly

Structured English	Pseudocode
Create a file and open it for writing	`OPENFILE <filename> FOR WRITE`
Open a file in append mode	`OPENFILE <filename> FOR APPEND`
Open a file for reading	`OPENFILE <filename> FOR READ`
Open a file for random access	`OPENFILE <filename> FOR RANDOM`
Close a file	`CLOSEFILE <filename>`
Write a record to a file	`PUTRECORD <filename>, <identifier>`
Read a record from a file	`GETRECORD <filename>, <identifier>`
Move to a specific disk address within the file	`SEEK <filename>, <address>`
Test for end of file	`EOF(<filename>)`

Table 26.01 Operations for file processing

Sequential file processing

If we have an array of records, we may want to store the records on disk before the program quits, so that we don't lose the data. We can open a binary file and write one record after another to the file. We can then read the records back into the array when the program is run again.

WORKED EXAMPLE 26.02

Processing records in a sequential file

Table 26.02 shows the pseudocode for storing the car records from Worked Example 26.01 in a sequential file and accessing them.

Saving contents of array	Restoring contents of array
`OPENFILE CarFile FOR WRITE`	`OPENFILE CarFile FOR READ`
`FOR i ← 1 TO MaxRecords`	`FOR i ← 1 TO MaxRecords`
`    PUTRECORD CarFile, Car[i]`	`    GETRECORD CarFile, Car[i]`
`NEXT i`	`NEXT i`
`CLOSEFILE CarFile`	`CLOSEFILE CarFile`

Table 26.02 Pseudocode for processing records

Processing records sequentially in Python, VB.NET and Java

Python

```
import pickle # this library is required to create binary files
Car = [CarRecord() for i in range(100)]

CarFile = open('CarFile.DAT', 'wb') # open file for binary write

for i in range(100):  # loop for each array element
   pickle.dump(Car[i], CarFile) # write a whole record to the binary file

CarFile.close()  # close file

CarFile = open('CarFile.DAT', 'rb')  # open file for binary read

Car = []  # start with empty list
while True:  # check for end of file
   Car.append(pickle.load(CarFile))  # append record from file to end of list

CarFile.close()
```

VB.NET

```
Option Explicit On
Imports System.IO

Dim CarFileWriter As BinaryWriter
Dim CarFileReader As BinaryReader
Dim CarFile As FileStream
Dim Car(100) As CarRecord ' declare an array of CarRecord type
Dim i As Integer

'link file to the filename
CarFile = New FileStream("CarFile.DAT", FileMode.Create)
' create a new file and open it for writing
CarFileWriter = New BinaryWriter(CarFile)
For i = 1 To 100  ' loop for each array element
   CarFileWriter.Write(Car(i).VehicleID)  ' write a field to the binary file
   CarFileWriter.Write(Car(i).Registration)
   CarFileWriter.Write(Car(i).DateOfRegistration)
   CarFileWriter.Write(Car(i).EngineSize)
   CarFileWriter.Write(Car(i).PurchasePrice)
```

(Continued)

```
Next

CarFileWriter.Close()   ' close file channel
CarFile.Close()

'link file to the filename
CarFile = New FileStream("CarFile.DAT", FileMode.Open)
' create a new file and open it for reading
CarFileReader = New BinaryReader(CarFile)
i = 0

' loop until end of binary file reached
i = 0
Do While CarFile.Position < CarFile.Length
    ' read fields from the binary file
   Car(i).VehicleID = CarFileReader.ReadString()
   Car(i).Registration = CarFileReader.ReadString()
   Car(i).DateOfRegistration = CarFileReader.ReadString()
   Car(i).EngineSize = CarFileReader.ReadInt32()
   Car(i).PurchasePrice = CarFileReader.ReadDecimal()
   i = i + 1
Loop

CarFileReader.Close()   'close file channel

CarFile.Close()
```

Java

writing records to a file:

```
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.EOFException;
import java.io.FileInputStream;
import java.io.FileOutputStream;
   try
   {  // set up file stream and link to file name
      FileOutputStream fos = new FileOutputStream("CarFile.DAT");
      // link file stream to data stream
      DataOutputStream dos = new DataOutputStream(fos);
      for (int i = 1; i < 100; i++) // loop for each array element
      {
         dos.writeUTF(car[i].vehicleID); // write a field to the file
         dos.writeUTF(car[i].registration);
         dos.writeInt(car[i].engineSize);
         dos.writeUTF(car[i].dateOfRegistration);
         dos.writeDouble(car[i].purchasePrice);
      }
      dos.close(); // close data stream
   }
   catch (Exception x)
   {
      System.out.println("IO error");
   }
```

reading records back into array:

```
   CarRecord[] car = new CarRecord[100];
   try
   {  // set up file stream and link to file name
      FileInputStream fis = new FileInputStream("CarFile.DAT");
      // link file stream to data stream
      DataInputStream dis = new DataInputStream(fis);
      int i = 1;
      while (true) // loop for each array element
      {
```


(Continued)

```
            thisCar = new CarRecord();
            thisCar.vehicleID = dis.readUTF(); // read fields from the file
            thisCar.registration = dis.readUTF();
            thisCar.engineSize = dis.readInt();
            thisCar.dateOfRegistration = dis.readUTF();
            thisCar.purchasePrice = dis.readDouble();
            car[i] = thisCar; // assign record to next array element
            i += 1;
        }
    }
    catch (EOFException x)
    {
        System.out.println("End of File reached");
    }
    catch (Exception x)
    {
        System.out.println(x);  // output error message
    }
```

Alternative method using Date data type (which is an object in Java):

write to file

```
import java.io.*;
import java.util.Date;

class CarRecord implements java.io.Serializable // to allow writing and reading
                                                // objects
{
    String vehicleID;
    String registration;
    Date dateOfRegistration;
    int engineSize;
    double purchasePrice;

    public CarRecord()    // declare constructor
    {
        vehicleID = "XX";
        registration = "";
        dateOfRegistration = new Date();
        engineSize = 0;
        purchasePrice = 0.00;
    }
}

class Program
{
public static void main(String[] args)
{
    CarRecord[] car = new CarRecord[100];
    CarRecord thisCar;

    try
    {   //Write array to file.

        FileOutputStream fos = new FileOutputStream("cars.ser");
```


(Continued)

```
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        for (int i = 1; i < 100; i++) // loop for each array element
        {
           oos.writeObject(car[i].vehicleID); // write a field to the file
           oos.writeObject(car[i].registration);
           oos.writeObject(car[i].engineSize);
           oos.writeObject(car[i].dateOfRegistration);
           oos.writeObject(car[i].purchasePrice);
        }
        oos.close();
    }
    catch (EOFException x)
    {
        System.out.println("End of File reached");
    }
      catch (Exception x)
    {
        System.out.println(x);
    }
}
}
```


reading file into array:

```
CarRecord[] car = new CarRecord[100];
try
{
   //Read array from file.
   FileInputStream fis = new FileInputStream("cars.ser");
   ObjectInputStream ois = new ObjectInputStream(fis);
   int i = 1;
   while (true)
   {
      car[i].vehicleID = (String) ois.readObject(); // read the first item from the
                                                    // file to the field
      car[i].registration= (String) ois.readObject();
      car[i].engineSize = (int) ois.readObject();
      car[i].dateOfRegistration = (Date) ois.readObject();
      car[i].purchasePrice = (double) ois.readObject();
      i += 1;
   }
}
catch (EOFException x)
{
   System.out.println("End of File reached");
}
catch (Exception x)
{
   System.out.println(x);
}
```

TASK 26.01

1 Write a complete program to save several car records to a sequential file.
2 Write another program to read the file and display the contents on screen.

Random-access file processing

Instead of storing records in an array, we may want to store each record in a binary file as the record is created. We can then update the record in situ (read it, change it and save it back in the same place). Note that this only works for fixed-length records. We can use a hashing function to calculate an address from the record key and store the record at the calculated address in the file (this is similar to using a hash table, see Chapter 23, Section 23.11). Just as with a hash table, collisions may occur and records need to be stored in the next free record space.

WORKED EXAMPLE 26.03

Processing records in a random-access file

Table 26.03 shows the pseudocode for storing a car record from Worked Example 26.01 in a random-access file and accessing it.

Saving a record	Retrieving a record
`OPENFILE CarFile FOR RANDOM`	`OPENFILE CarFile FOR RANDOM`
`Address ← Hash(ThisCar.VehicleID)`	`Address ← Hash(ThisCar.VehicleID)`
`SEEK CarFile, Address`	`SEEK CarFile, Address`
`PUTRECORD CarFile, ThisCar`	`GETRECORD CarFile, ThisCar`
`CLOSEFILE CarFile`	`CLOSEFILE CarFile`

Table 26.03 Pseudocode for random-access file operations

`SEEK` moves a pointer to the given record address. The `PUTRECORD` and `GETRECORD` commands access the record at that address. After the command has been executed the pointer points to the next record in the file.

Processing random-access records in Python, VB.NET and Java

Python

```
import pickle  # this library is required to create binary files
ThisCar = CarRecord()

CarFile = open('CarFile.DAT','rb+')  # open file for binary read and write
Address = hash(ThisCar.VehicleID)
CarFile.seek(Address)
pickle.dump(ThisCar, CarFile)  # write a whole record to the binary file
CarFile.close()  # close file
```

to find a record from a given VehicleID:

```
CarFile = open('CarFile.DAT','rb')  # open file for binary read
Address = hash(VehicleID)
CarFile.seek(Address)
ThisCar = pickle.load(CarFile)  # load record from file
CarFile.close()
```

In Python, the hash function needs to allow for the record size in bytes. For example, if the record size is 58 bytes, then the second record slot starts at position 59. The *n*th record slot starts at position $(n - 1) \times 58 + 1$.

(Continued)

VB.NET

```
Dim CarFileWriter As BinaryWriter
Dim CarFileReader As BinaryReader
Dim CarFile As FileStream
Dim ThisCar, MyCar As CarRecord
' link the file to the filename
CarFile = New FileStream("CarFile.DAT", FileMode.Open)

' create a new file and open it for writing
CarFileWriter = New BinaryWriter(CarFile)
' get starting address for record
CarFile.Position = Hash(ThisCar.VehicleID)

' write fields to the binary file

CarFileWriter.Write(ThisCar.VehicleID)
CarFileWriter.Write(ThisCar.Registration)
CarFileWriter.Write(ThisCar.DateOfRegistration)
CarFileWriter.Write(ThisCar.EngineSize)
CarFileWriter.Write(ThisCar.PurchasePrice)

CarFileWriter.Close()  'close file channel
CarFile.Close()
```

to find a record from a given VehicleID:

```
CarFile = New FileStream("CarFile.DAT", FileMode.Open)
CarFileReader = New BinaryReader(CarFile)
' get starting address for record
CarFile.Position = Hash(VehicleID)

' read fields from the binary file
MyCar.VehicleID = CarFileReader.ReadString()
MyCar.Registration = CarFileReader.ReadString()
MyCar.DateOfRegistration = CarFileReader.ReadString()
MyCar.EngineSize = CarFileReader.ReadInt32()
MyCar.PurchasePrice = CarFileReader.ReadDecimal()

CarFileReader.Close()  'close file channel
CarFile.Close()
```

In VB.NET, the hash function needs to allow for the record size in bytes. For example, if the record size is 58 bytes, then the second record slot starts at position 59. The *n*th record slot starts at position $(n - 1) \times 58 + 1$.

Java

```
CarRecord thisCar;
int recordSize = 50;
try // set up a file with 100 dummy records
{
   RandomAccessFile writer = new RandomAccessFile("CarFile.DAT", "rw");
   for (int i = 1; i < 100; i++) // loop for each array element
   {
      thisCar = new CarRecord();
      thisCar.vehicleID = "A" + i;
      writer.seek(i * recordSize);
      writer.writeUTF(thisCar.vehicleID);
      writer.writeUTF(thisCar.registration);
      writer.writeUTF(thisCar.dateOfRegistration);
      writer.writeInt(thisCar.engineSize);
      writer.writeDouble(thisCar.purchasePrice);
   }
```

(Continued)

Java

```
    writer.close();
}
catch (IOException x)
{
}
```

to find a record from a given VehicleID:

```
try
{
    RandomAccessFile writer = new RandomAccessFile("CarFile.DAT", "rw");
    RandomAccessFile reader = new RandomAccessFile("CarFile.DAT", "r");
    reader.seek(hash(vehicleID));
    thisCar = new CarRecord();
    thisCar.vehicleID = reader.readUTF();
    thisCar.registration = reader.readUTF();
    thisCar.engineSize = reader.readInt();
    thisCar.dateOfRegistration = reader.readUTF();
    thisCar.purchasePrice = reader.readDouble();
    reader.close();
}
catch (IOException x)
{
}
```


TASK 26.02

Write a complete program to save several car records to a random-access file. Write another program to find a record in the random-access file using the record key. Display the record data on screen.

26.03 Exception handling

Run-time errors can occur for many reasons. Some examples are division by zero, invalid array index or trying to open a non-existent file. Run-time errors are called 'exceptions'. They can be handled (resolved) with an error subroutine (known as an 'exception handler'), rather than let the program crash.

Using pseudocode, the error-handling structure is:

```
TRY
    <statementsA>
EXCEPT
    <statementsB>
ENDTRY
```

Any run-time error that occurs during the execution of `<statementsA>` is caught and handled by executing `<statementsB>`. There can be more than one `EXCEPT` block, each handling a different type of exception. Sometimes a `FINALLY` block follows the exception handlers. The statements in this block will be executed regardless of whether there was an exception or not.

VB.NET is designed to treat exceptions as abnormal and unpredictable erroneous situations. Python is designed to use exception handling as flow-control structures. You may find you need to include exception handling in the code for Worked Example 26.02. Otherwise the end of file is encountered and the program crashes.

Python distinguishes between different types of exception, such as:

- `IOError`: for example, a file cannot be opened
- `ImportError`: Python cannot find the module
- `ValueError`: an argument has an inappropriate value
- `KeyboardInterrupt`: the user presses Ctrl+C or Ctrl+Del
- `EOFError`: a file-read meets an end-of-file condition
- `ZeroDivisionError`: a division by zero has been attempted

Java distinguishes between different types of exception, such as:

- `IOException`: for example, a file cannot be opened
- `ArithmeticException`: an arithmetic error occurred, such as division by zero
- `ArrayIndexOutOfBoundsException`: the program tries to access an array element outside the boundary

WORKED EXAMPLE 26.04

Here is a simple example of exception handling. Asking the user to key in an integer could result in a non-integer input. This should not crash the program.

Python

```
NumberString = input("Enter an integer: ")
try:
    n = int(NumberString)
    print(n)
except:
    print("This was not an integer")
```

VB.NET

```
Dim NumberString As String
Dim n As Integer
Console.WriteLine("Enter an integer")
NumberString = Console.ReadLine()
Try
    n = Int(NumberString)
    Console.WriteLine(n)
Catch
    Console.WriteLine("This was not an integer")
End Try
```

Java

```
import java.util.Scanner;
{
   public static void main(String[] args)
   {
      Scanner console = new Scanner(System.in);
      try
      {
         System.out.print("Enter an integer: ");
         int n = console.nextInt();
         System.out.println(n);
      }
      catch(Exception e) // catches any exception
      {
         System.out.println("This was not an integer");
      }
   }
}
```

TASK 26.03

Add exception-handling code to your programs for Task 26.01 or Task 26.02. Test your code handles exceptions without the program crashing.

Reflection Point:

Can you explain the difference between serial, sequential and random access files? Give an example of when each is appropriate.

Summary

- Records are user-defined types.
- Records can be stored in files in a serial, sequential or random (direct access) manner.
- Exception handling is advisable to avoid program crashes due to run-time errors.

Exam-style Questions

1 A company stores details about their customers in a binary file of records.

- The key field of a customer record is the customer ID (a number between 100001 and 999999).
- The name of the customer is stored in a 30-character field.
- The customer's telephone number is stored in a 14-character field.
- The total value of orders so far is stored in a currency (decimal) field.

a **i** Write the declaration statement for the record data type `CustomerRecord` required to store the data. Write program code. [6]

ii Write the declaration statement for an array `CustomerData[0 : 999]` to store customer records. [2]

b The array `CustomerData` is to be used as a hash table to store customer records. The function `Hash` is used to calculate the address where a record is to be stored.

```
FUNCTION Hash(CustomerID : INTEGER) RETURNS INTEGER
    Address ← CustomerID MOD 1000
    RETURN Address
ENDFUNCTION
```

i Write program code to implement the function `Hash`. [3]

ii Write a procedure `AddRecord(Customer : CustomerRecord)` to add a customer record to the hash table `CustomerData`. Your solution should handle collisions by using the next available slot in the hash table. [7]

iii Write a function `FindRecord(CustomerID : INTEGER)` that returns the index of the hash table slot where the record for the customer with `CustomerID` is stored. [7]

c Before the program stops, the hash table records must be stored in a sequential file, so that the records can be restored to the array when the program is re-entered.

Write program code to store the records of the array CustomerData sequentially into a binary file CustomerData.DAT [6]

d Instead of using a hash table, the company decide they want to store customer records in a direct-access binary file.

Explain what changes need to be made to your program to do this. [6]

2 A program allows a user to enter a filename for accessing a data file. If the user types in a filename that does not exist, the program crashes. Write program code that includes exception handling to replace the following pseudocode: [5]

```
OUTPUT "Which file do you want to use? "
INPUT FileName
OPENFILE FileName FOR RANDOM
```